Fiche d'exercices
Mini-Projet
Candy Crush

1. Introduction

Candy Crush est un jeu vidéo dont l'objectif est de faire des combinaisons de bonbons afin de marquer des points. La grille de jeu est constituée de cases, chacune contenant un bonbon de couleur. Le joueur peut échanger la position de deux bonbons voisins dans la grille si l'échange crée une combinaison d'au moins 3 bonbons alignés de même couleur, qui sera alors automatiquement supprimée. Les bonbons tombent alors dans la grille pour combler les trous laissés par les bonbons supprimés et de nouveaux bonbons entrent dans la grille par le haut pour combler toutes les cases vides (voir Figure 1). Il est possible que la descente des bonbons créée de nouvelles combinaisons qui seront alors supprimées et ainsi de suite jusqu'à ce que la grille ne contienne plus de combinaison. Le joueur peut alors procéder à un nouvel échange de deux bonbons. Le joueur marque des points pour chaque combinaison de bonbons supprimée, le but du jeu étant de marquer le plus de points possible. Le jeu s'arrête lorsqu'il n'est plus possible d'échanger deux bonbons.

L'objectif du projet est de coder le jeu Candy Crush.

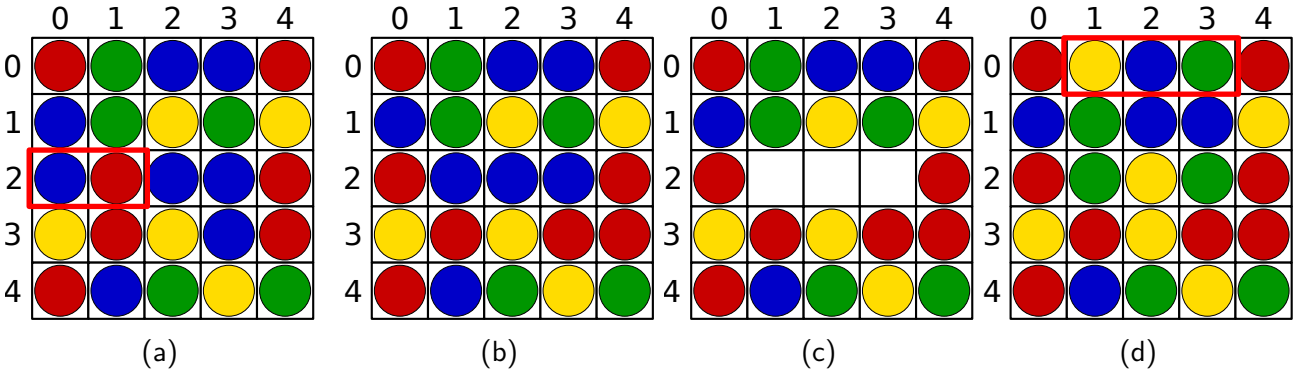

Figure 1 – Illustration d'un tour de jeu du Candy Crush. (a) Une grille contenant 4 types de bonbons différents (rouge, vert, bleu et jaune). Le joueur sélectionne deux bonbons voisins de coordonnées (2,0) et (2,1) à échanger (encadré rouge). (b) Résultat de la grille (a) après échange des deux bonbons. Trois bonbons bleus sont alignés horizontalement et constituent donc une combinaison. (c) Les bonbons de la combinaison sont supprimés. (d) les bonbons tombent pour combler les trous et 3 nouveaux bonbons (encadré rouge) sont créés pour compléter la grille.

2. Cahier des charges

Votre projet devra respecter le cahier des charges suivant :

- La grille de jeu sera représentée par une liste 2D d'entiers. Un bonbon sera donc représenté par un entier dans cette grille.
- Vous devrez choisir un des niveaux de difficulté de la règle du jeu défini dans la section 3.

— Votre propre code ne devra utiliser aucune autre bibliothèque que la bibliothèque standard de Python (pas de Numpy, Matplotlib limité à l'affichage de la grille...). **Seule cette première version de code sera à déposer sur moodle à la fin de la séance 4 et évaluée**. Libre à vous par la suite de la faire évoluer en intégrant des bibliothèques (prévoyez un nouveau fichier) .

— Votre code ne devra pas faire appel à de la récursivité [1]

— Votre programme sera capable de lire un fichier CSV décrivant une grille et de lancer le jeu sur cette grille. Ce fichier ne contient que des codes couleurs de bonbons, séparés par des espaces. On supposera que les couleurs possibles pour cette grille vont de 0 à la valeur maximale indiquée dans la grille. Le fichier aura la même structure que le fichier `exemple_grille.csv` fourni. Vous apprendrez à manipuler des fichiers csv lors de la séance 2 bis.

— L'arrêt de la boucle principale de votre programme pourra se faire soit au bout d'un certain nombre d'itérations, soit lorsqu'il n'y a plus d'échange possible. Pour cela, écrivez une fonction `candy_crush` qui lance le jeu à partir d'une grille mémorisée dans un fichier, *affiche l'état de la grille à chaque tour*, et s'arrête s'il n'y a plus d'échange possible pour la personne qui joue ou si un nombre de tours donné est atteint.

```
1  def candy_crush(un_fichier, nb_iter):
2      """
3        Jeu de Candy-crush à partir d'une grille mémorisée dans le fichier de nom
            un_fichier (chaine de caractères). Arrêt s'il n'y a plus d'échange
            possible ou si le nombre de tours nb_iter (entier) a été atteint.
4        Cette fonction ne retourne rien et affiche l'état de la grille à chaque
            tour.
5      """
```

— Votre programme devra utiliser une fonction `affichage_grille` permettant d'afficher la grille de jeu au joueur.

Durant le développement de votre projet, **il est indispensable de coder votre propre fonction afin qu'elle affiche la grille de jeu dans la console Spyder**. Cela facilite le deboggage. Lorsque votre code sera fonctionnel, vous pouvez utiliser le code suivant afin d'avoir une visualisation graphique de votre grille de jeu.

```
1  # Ajouter l'import suivant en début de programme:
2  # import matplotlib.pyplot as plt
3  def affichage_grille(grille, nb_type_bonbons):
4      """
5          Affiche la grille de jeu "grille" contenant au maximum nb_type_bonbons (
              entier) couleurs de bonbons différentes. Les bonbons sont codés entre
               0 et nb_type_bonbons-1.
6      """
7      plt.imshow(grille, vmin=0, vmax=nb_type_bonbons-1, cmap='jet')
8      plt.pause(0.1)
9      plt.draw()
10     plt.pause(0.1)
```

1. La récursivité sera vu au prochain chapitre. Si vous ne connaissez pas ce mot, alors c'est que vous ne le ferez pas. Donc pas de soucis, vous pouvez juste ignorer ce point.

En dehors de ces règles, vous pouvez ajouter des fonctionnalités à votre projet à votre guise. Nous vous conseillons néanmoins de réaliser une première version fonctionnelle sans ajout de fonctionnalités (et de la conserver dans un fichier à part nommé v1), puis d'étoffer par la suite votre projet si vous en avez le temps.

3. Niveaux de difficulté

On définit une combinaison par au minimum 3 bonbons de même valeur alignés horizontalement ou verticalement. La suppression d'une combinaison peut se faire de différentes manières. Nous vous proposons 2 niveaux de difficulté, illustrés sur la figure 2.

3.1 Niveau 1

Dans ce niveau, on supprime tous les bonbons de même valeur qui sont alignés horizontalement ou verticalement, avec un minimum de 3 bonbons. Les saisies de l'utilisateur sont supposées correctes.

3.2 Niveau 2

Dans ce niveau, si au moins 3 bonbons sont alignés (verticalement ou horizontalement), alors on supprime tous les bonbons de même valeur qui sont voisins d'un bonbon supprimé (un des 3 bonbons d'origine, ou un voisins de voisin).

4. Déroulement du projet

Ce projet est à faire en groupe de 3 personnes et se déroule sur 4 séances de TD + 1 séance d'évaluation. Nous vous conseillons le déroulement suivant :

- **Séance 1 : Découpage fonctionnel**
  - Proposez un algorithme en français, puis identifiez les différents sous-problèmes du Candy Crush. Faites valider cette étape par votre enseignant (∼30/45min).
  - Rédigez votre découpage fonctionnel sur un logiciel de traitement de texte ou dans un fichier Python.
  - Déposez votre découpage fonctionnel sur Moodle dans « Rendu de la décomposition fonctionnelle ».
- **Séance 2 : Répartition des tâches et programmation**
  - Répartissez les différentes fonctions et le programme principal à coder entre les membres du groupe
  - Selon les conseils de votre enseignant ou enseignante, choisissez un mode de fonctionnement pour partager le code entre vous : utilisation de Markolla, copie dans un cloud, logiciel de gestion de versions type git (cf. Section Travail collaboratif de la page ressource commune (Python) sur Moodle...)
  - Implémentation : codage et test des fonctions écrites.
- **Séance 2 bis (1h) : Introduction aux fichiers CSV**
  - Faites les exercices de la séance
  - Réfléchissez à l'utilisation pour votre projet.

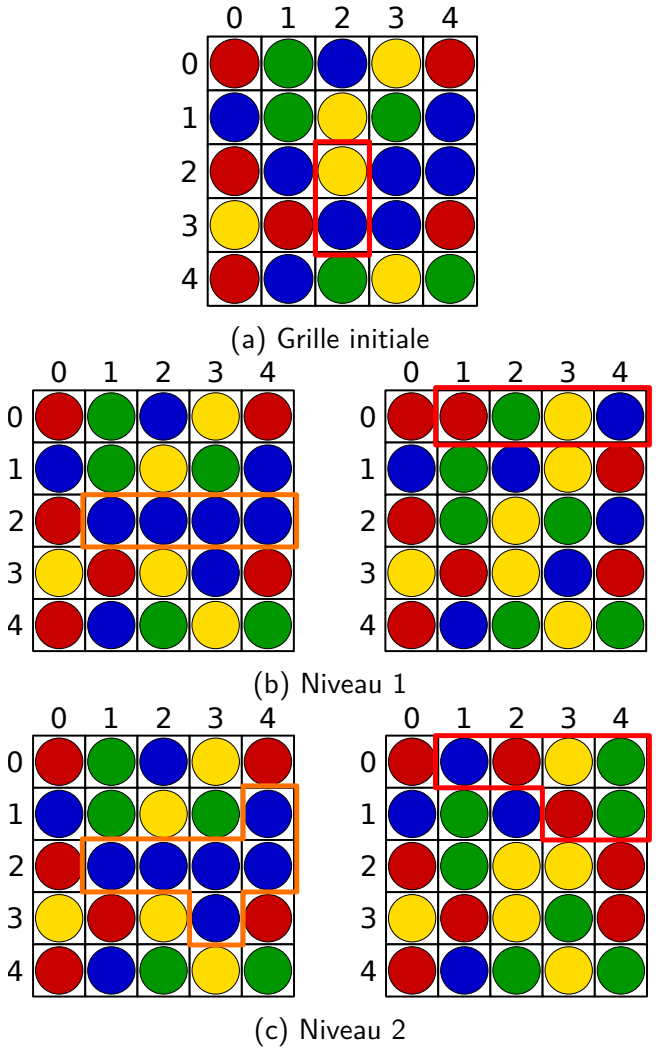

FIGURE 2 – Illustration des différents niveaux de difficultés. (a) la grille initiale où le joueur choisit d'échanger deux bonbons (encadré rouge). (b-d) les 2 niveaux de difficulté. Les grilles de gauche montrent le résultat après échange des bonbons où l'encadré orange entoure les bonbons qui vont être supprimés. Les grilles de droite montrent le résultat après suppression des bonbons, où l'encadré rouge entoure les bonbons qui ont été créés.

- **Séance 3 : Programmation et complexité**
  - Implémentation : codage et test des fonctions écrites[2].
  - Vérifiez entre membres d'une même équipe que vous partagez toujours les mêmes signatures de fonctions.
  - Choisissez une tâche (non triviale) pour laquelle vous vous calculerez la complexité et faites-la valider par votre professeur.
- **Séance 4 : Finalisation et rendu**
  - Implémentation : codage et test des fonctions écrites.
  - Rédaction du document à rendre
  - Dépôt des différents rendus sur Moodle
- **Séance 5 : Évaluation par les pairs**
  - Évaluez le travail de vos pairs, avec sérieux et bienveillance, selon le mode indiqué en section 6.

2. Pensez à tester chaque fonction et à conserver le code de vos tests dans des fichiers séparés.

5. Consignes de rendu

Pour tous les dépôts : un seul rendu par groupe.

- **Fin de la séance 1 :**
  Vous devez déposer sur Moodle un fichier contenant le découpage fonctionnel que vous envisagez de mettre en place ou son lien markolla. Ce découpage fonctionnel ne sera pas évalué, mais le rapport à rendre en fin de séance 4 devra contenir votre algorithme principal en français mis au propre.
- **Fin de la séance 3 :**
  Vous devez déposer sur Moodle une archive (dossier compressé) contenant vos tests du code (une ou plusieurs fonctions) qui vous permet d'identifier les alignements dans une grille (cf cours, partie sur le test de fonctions).
- **Fin de la séance 4 :**
  Vous devez déposer sur Moodle dans l'activité *Rendu projets et évaluation par les pairs* :
  - Le document *Modèle de rapport* complété au format pdf (disponible sur Moodle)
  - L'archive de tests de la détection des alignements
  - Le code final de votre projet

6. Évaluation

Ce projet va être évalué par vos camarades à travers un système d'évaluation par les pairs. Chaque groupe recevra, lors de la dernière séance, 2 projets qu'il devra évaluer selon une grille critériée prédéfinie, disponible sur Moodle. L'attribution des projets se fera aléatoirement et anonymement, et l'évaluation se déroulera directement sur Moodle. À la fin de la séance, vous recevrez donc 2 évaluations différentes vous permettant de situer votre projet par rapport à ce qui été attendu.

Remarque : La "note" obtenue au terme de ce projet ne comptera pas dans votre moyenne. En revanche, des questions proches du sujet du projet pourront-être posées lors de l'IE d'Avril.